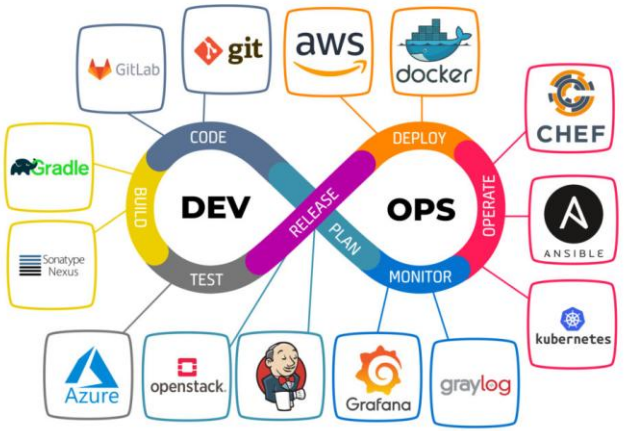
TEK-UP
Ecole Supérieure Privée
Technologie & Ingénierie

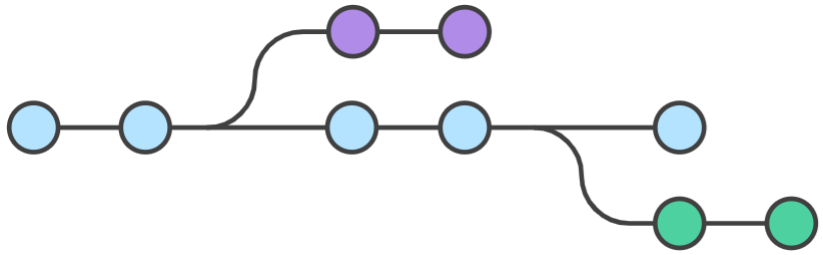


Git & GitHub

CHAPITRE 3

NOTION DE BRANCHES

GIT MERGE



Chapitre 3
Notion de branches

Récap sur Git

- **Pour comparer le répertoire de travail avec la zone d'index après les modifications**

$ echo 'diff newline' >> file_name

$ git status

$ git diff file_name // pour voir quelle modification est ajoutée ou supprimée

- **Pour comparer les commits dans git**

$ git log --oneline

$ git diff older_commit new_commit // pour détecter exactement ce qui a été récemment ajouté à older_commit

$ git diff commitA commitB --name-only // pour ne lister que le nom des fichiers impactés

Récap sur Git

Annuler les modifications

Roll Back est sur le point d'annuler les modifications que nous avons faites dans le dépôt

Dans GIT, cela peut être fait via RESET et REVERT.

➤ RESET - La commande git reset renvoie l'environnement local à un commit précédent.

➤ REVERT - La commande git revert est similaire à reset, mais son approche est différente.

- Elle ajoute un nouveau commit à la fin de la chaîne pour "annuler" les changements.

➤ RESET ou REVERT ?

- Si l'utilisateur a déjà envoyé (push) des commits au dépôt distant, un revert est un moyen plus intéressant pour annuler les modifications.

- Si Commit in local alors Reset est le mieux pour l'annulation, Si commit est pushed alors revert sera la bonne option.

Récap sur Git

Annuler les modifications git reset --hard

Pour effectuer “ git reset --hard “ de nos modifications, on procède comme suit :

$ echo 'nouveau code' > file_reset.txt

$ git add .

$ git commit -m "commit to reset"

$ git push

$ git log --pretty=oneline --decorate // Imprimer les noms de référence des commits affichés.

$ git reset --hard id_of_commit // ou en utilisant $ git reset --hard HEAD~N (N représente le nombre de commits que nous souhaitons supprimer)

$ git log --oneline // il affiche toutes les historiques des commits pour vérifier que nous avons bien réinitialisé notre commit

Récap sur Git

Annuler les modifications git reset --soft

Pour effectuer "git reset --soft" de nos modifications, on peut utiliser :

$ git reset --soft id_of_commit

$ git status // notez qu'il y a des fichiers prêts à être commités et en attente dans la zone d'index

$ git reset file_name // pour annuler l'ajout de git et laisser les fichiers non suivis par git

$ git status // pour vérifier que vos fichiers se trouvent maintenant dans la zone d'index.

➔ Nous pouvons corriger notre erreur en modifiant nos fichiers, puis ajouter-les, valider-les et envoyer-les

➔ git add, git commit et git push pour terminer le travail

Récap sur Git

Annuler les modifications git revert

Pour annuler "git revert" vos modifications, procédez comme suit :

$ git revert HEAD // permet d'ignorer votre dernier commit et placera un nouveau commit dessus

$ git revert --no-commit HEAD // pour vérifier quels fichiers vont être modifiés/supprimés avant de confirmer la validation (commit)

$ git revert --no-commit HEAD~3.. // pour ignorer les trois derniers commits à la fois

$ git revert --no-commit xyz..HEAD // pour ignorer tous les commits : de commit xyz jusqu'à HEAD

$ git revert --abort // pour annuler le retour

$ git revert --continue // pour valider le retour et l'écriture du message de commit

Git : Notion de branches

Branche de production Main **ou** Master

Le principal avantage de Git est **son système de branches**.

➔ C'est sur ces branches que repose tous les avantages de Git !

➔ Les différentes branches correspondent à des copies de votre code principal à un instant T, où vous pourrez tester toutes les modifications sans impacter le code principal.

➔ Sous Git, la branche principale est appelée la **branche main**, ou **master** pour les dépôts créés.

➔ GitHub a opéré un changement de terme de "master" à "main" en octobre 2020 (les deux termes sont retenus).

➔ Il faut voir les branches comme autant des dossiers différents.

Git : Notion de branches

Branche de production Main **ou** Master

- La création de branches est une fonctionnalité disponible dans la plupart des systèmes de contrôle de version.

- Le fait de créer une branche à part consiste à créer un **pointeur virtuel pointant sur toutes les opérations effectuées au niveau de cette branche**.

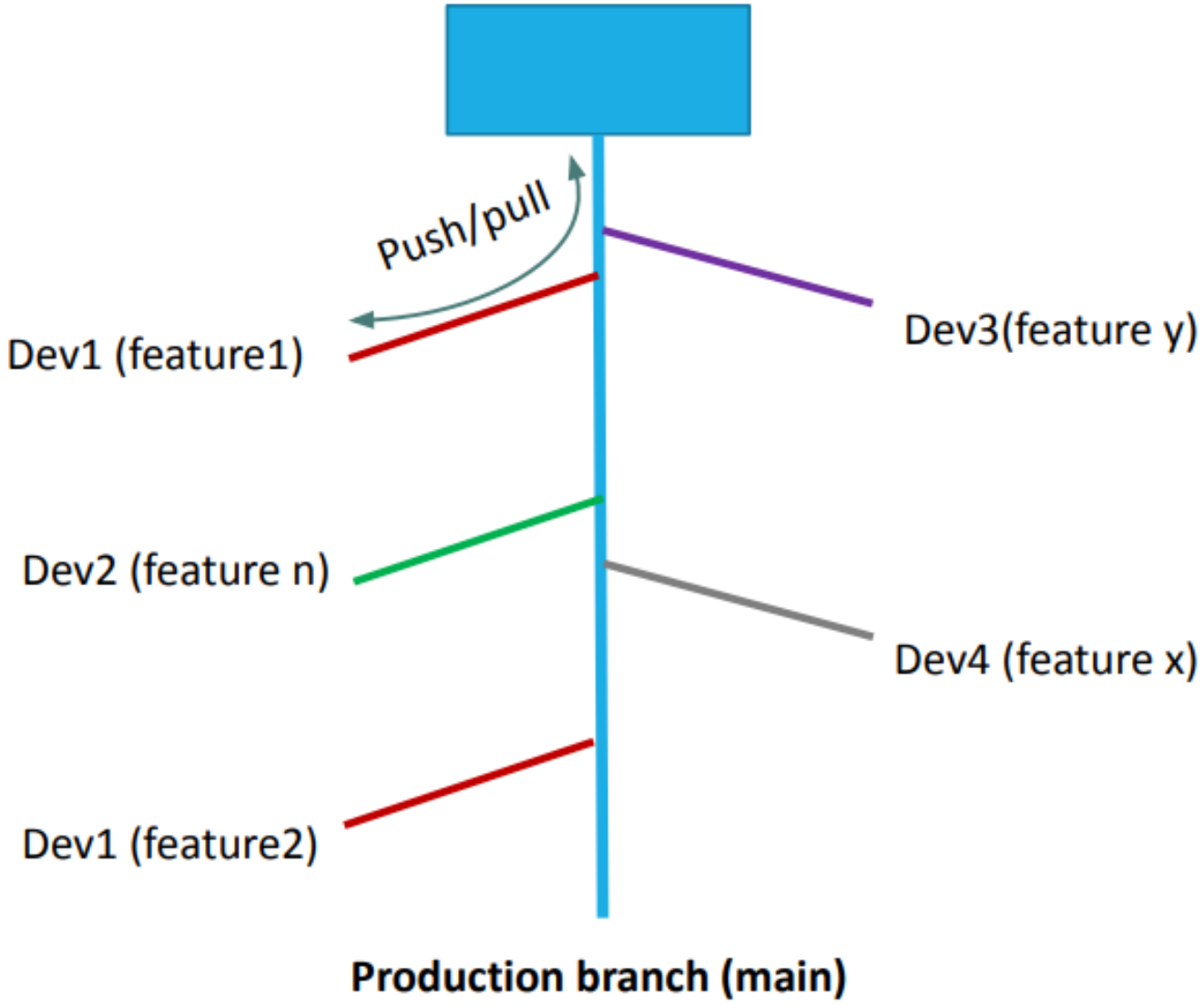
- Par exemple, si un développeur voulait développer une fonction, il le fait ressortir dans une branche à part.

- Ainsi, la fonction sera codée et testée au niveau de la branche sans toucher à la branche principale.

- Après avoir assuré qu'elle fonctionne correctement, on pourra la fusionner dans la branche principale (main, master).

Git : Notion de branches

Branche de production Main ou Master



Git : Notion de branches

Comment fonctionne ?

- Une branche représente une ligne de développement indépendante.
- Elle peut être considérée comme un moyen de créer un tout nouveau répertoire de travail, une zone de préparation et un historique du projet.
- Les nouveaux commits sont enregistrés dans l'historique de la branche actuelle, ce qui entraîne un embranchement dans l'historique du projet.
- La commande git branch nous permet de créer, répertorier, renommer et supprimer des branches.
- Il ne nous permet pas de basculer entre les branches.
- Pour cette raison, une commande git checkout et git merge seront utiles.

!! **git merge vous permet de sélectionner les lignes de développement indépendantes créées avec git branch et de les intégrer à une seule branche**

Git : Notion de branches

Créer des branches

- Il est important de comprendre que les branches ne sont que des pointeurs vers des commits.
- Lorsque nous créons une branche, tout ce que Git a à faire est de créer un nouveau pointeur, sans changer le dépôt.
- Nous démarrons avec un dépôt qui ressemble à ceci :



Git : Notion de branches

Branche de production Main **ou** Master

- Ensuite, nous allons créer une branche en utilisant la commande :

$ git branch branche1

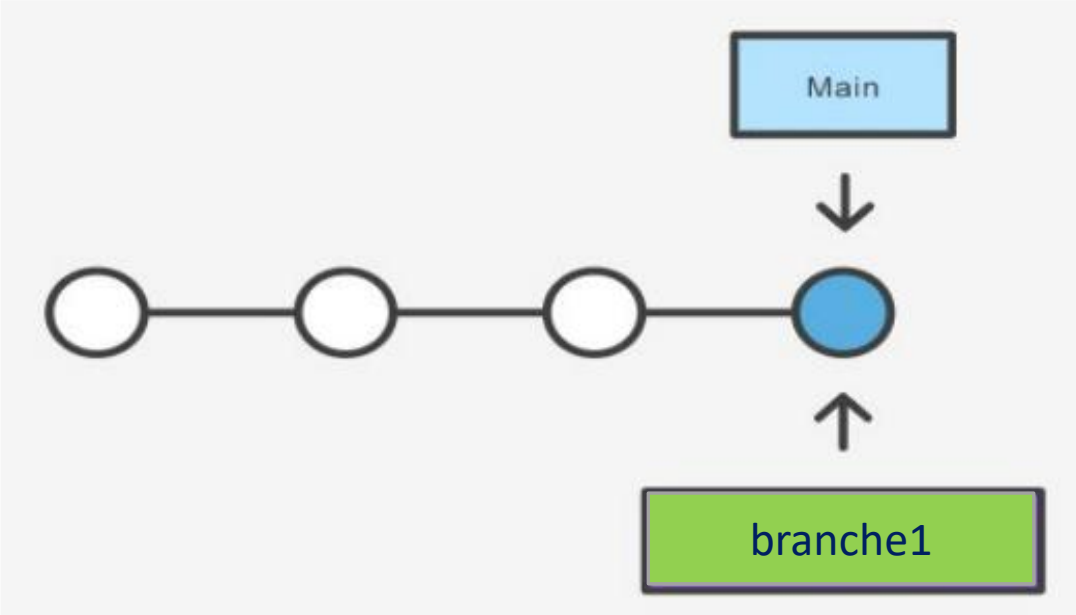
L'historique du dépôt reste inchangé. Tout ce que nous obtenons est un nouveau pointeur vers le commit

- Notons que cela ne crée que la nouvelle branche.
- Pour commencer à y ajouter des commits,

nous devons le sélectionner avec git checkout,

puis créer des modifications et utiliser les

commandes standard git add et git commit.



Git : Notion de branches

Git Checkout

❖ Dans Git, un "checkout" permet de basculer entre différentes versions d'une entité ciblée.

❖ La commande git checkout opère sur trois entités distinctes : les fichiers, les validations (commits) et les branches.

❖ Dans le sujet Annulation des modifications, nous avons vu comment git checkout affiche les anciens commits.

❖ L'extraction des branches est similaire à l'extraction des anciens commits et fichiers dans la mesure où le répertoire de travail est mis à jour pour correspondre à la branche (révision sélectionnée).

➔ Cependant, les nouvelles modifications sont enregistrées dans l'historique du projet, c'est-à-dire qu'il ne s'agit pas d'une opération en lecture seule.

Git : Notion de branches

Vérification des branches

❖ La commande git checkout permet de **naviguer** entre les branches créées par git branch.

❖ L'extraction d'une branche met à jour les fichiers du répertoire de travail pour correspondre à la version stockée dans cette branche, et indique à Git d'enregistrer tous les nouveaux commits sur cette branche.

❖ La commande git checkout peut parfois être confondue avec la commande git clone.

➔ La différence entre ces deux commandes est que clone fonctionne pour **récupérer le code** d'un dépôt distant, alternativement checkout fonctionne pour **basculer entre les versions de code** déjà sur le système local.

Git : Notion de branches

Utilisation : Branches existantes

❖ En supposant que le dépôt dans lequel nous travaillons contient des branches préexistantes, nous pouvons basculer entre ces branches à l'aide de git checkout.

❖ Pour savoir quelles branches sont disponibles et quel est le nom de la branche actuelle, nous pouvons exécuter git branch.

```
$  git branch
master
Branche1
$ git checkout branche1
```

❖ Cet exemple montre comment afficher une liste des branches disponibles en exécutant la commande git branch et passer à une branche spécifiée, dans ce cas, le branche1

Git : Notion de branches

Création de branche distante

- Les exemples précédents ont tous démontré des opérations de branches locales.
- La commande git branch fonctionne également sur les branches distantes.
- Pour opérer sur des branches distantes, un dépôt distant doit être configuré et ajouté à la configuration du dépôt local.

```
$ git remote add origin https://bitbucket.com/user/repo.git
```

Ajouter un dépôt distant à la configuration du dépôt local

```
$ git push <origin> branche1~ # pusher la branche branche1 vers origin
```

➔ Cette commande poussera une copie de la branche locale branche1 vers le dépôt distant <remote>.

Git : Notion de branches

Quelques commandes utiles

$ git branch --list //Répertoriez toutes les branches de dépôt.

$ git branch <branch> // Créer une nouvelle branche appelée <branch>.

$ git branch -d <branch_name> // Supprimer la branche spécifiée, Git vous empêche de supprimer la branche si elle contient des modifications non fusionnées

$ git branch -D <branch_name> // Forcer la suppression de la branche spécifiée, même si elle contient des modifications non fusionnées.

$ git branch -m <branch> // Renommer la branche actuelle en <branch>.

$ git branch –m old_name new_name // Renommer la branche.

$ git branch –a // Répertorier toutes les branches distantes (Pour lister toutes les branches).

$ git checkout branch_name // Pour passer à une branche spécifique

Git : Notion de branches

Quelques commandes utiles

$ git push origin --delete name_branch // Pour supprimer une branche distante.

Ou bien :

$ git push origin : name_branch // Pour supprimer une branche distante.

➔ Cela enverra un signal de suppression au dépôt origin distant qui déclenchera une suppression de la branche distante.

$ git checkout -b "branch_name" // créer et switchez dans une branche à la fois

Git : Notion de branches

Manipulation des étapes de branche :

1. $ git branch // Pour voir la branche par défaut (master)
2. $ git branch feature1 // Pour créer une nouvelle branche (feature1)
3. $ git branch // Pour voir les branches (master, feature1)
4. $ git checkout feature1 // Pour basculer de : master → feature1
5. $ git branch // Pour voir ou il s'est pointé (feature1)

➔ Pour créer une branche et y basculer directement, nous pouvons utiliser :

$ git checkout –b feature2

6. Essayons de créer différents fichiers dans ces branches et comparons-les avec la branche principale

➔ On cherche à envoyer cette nouvelle branche, nous utilisons $ git push origin feature

➔ Mais, avant d'envoyer cette branche à distance, $ git branch --set-upstream-to=origin/feature

Chapitre 3
Git merge

Récap sur Git

Créer des branches et basculer vers la branche désirée

On peut créer des branches, opérer (créer des fichiers, indexer et commiter) et enregistrer les historiques de chaque branches séparément :

$ git branch nom_branche// permet de créer une branche « nom_branche »

$ git checkout nom_branche // pour basculer vers la branche créée

$ git checkout –b branche2 // pour créer et basculer vers la branche2 à la fois

$ git branch --list // pour lister toutes les branches

$ git branch // pour afficher où pointe le HEAD

$ git fetch --all // pour faire le checkout d'une branche distante, il faut d'abord faire un fetch du contenu de cette branche (aller vers la branche, télécharger des commits, des fichiers et des réfs d'un dépôt distant vers votre dépôt local).

$ git remote add origin <lien de dépôt distant> // pour établir le lien entre dépôt local et dépôt distant

$ git push origin branche2 // pour envoyer la banche2 avec tout son historique vers dépôt distant

Git : Notion de fusion

Comment fusionner mes branches

- La commande git merge nous permet de sélectionner les lignes de développement indépendantes créées avec git branch et de les **intégrer à une seule branche**.
- Toutes les opérations et les commandes utilisées après git merge font un merge dans la branche **courante**.
- La branche courante sera mise à jour pour faire état du merge, tandis que la branche cible restera inchangée.

➔ Cela signifie que la commande git merge est souvent utilisée en association avec git checkout pour sélectionner la branche courante

➔ git branch -d // pour supprimer la branche cible obsolète (qui n'est plus en usage).

Git Merge

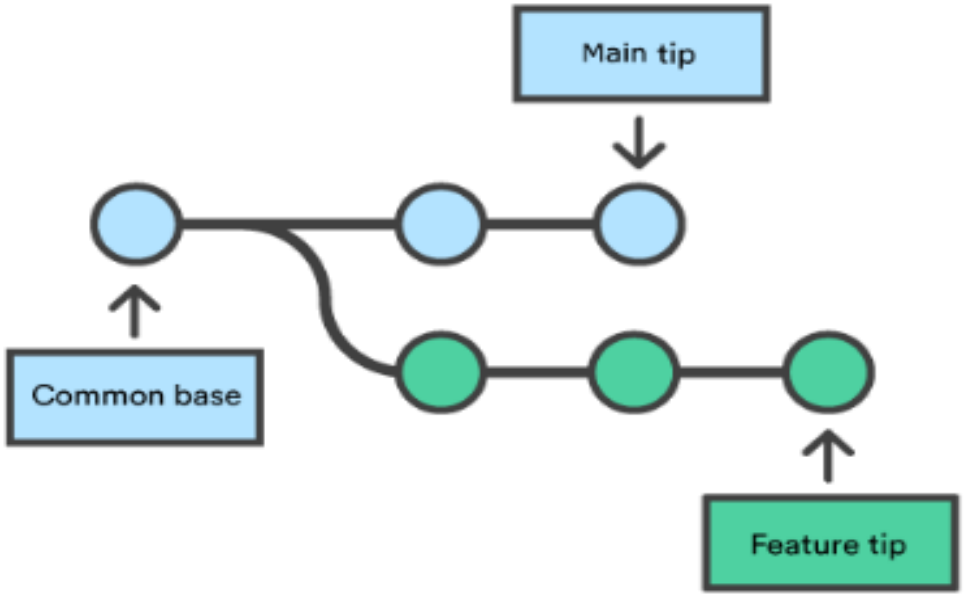
Fonctionnement (1)

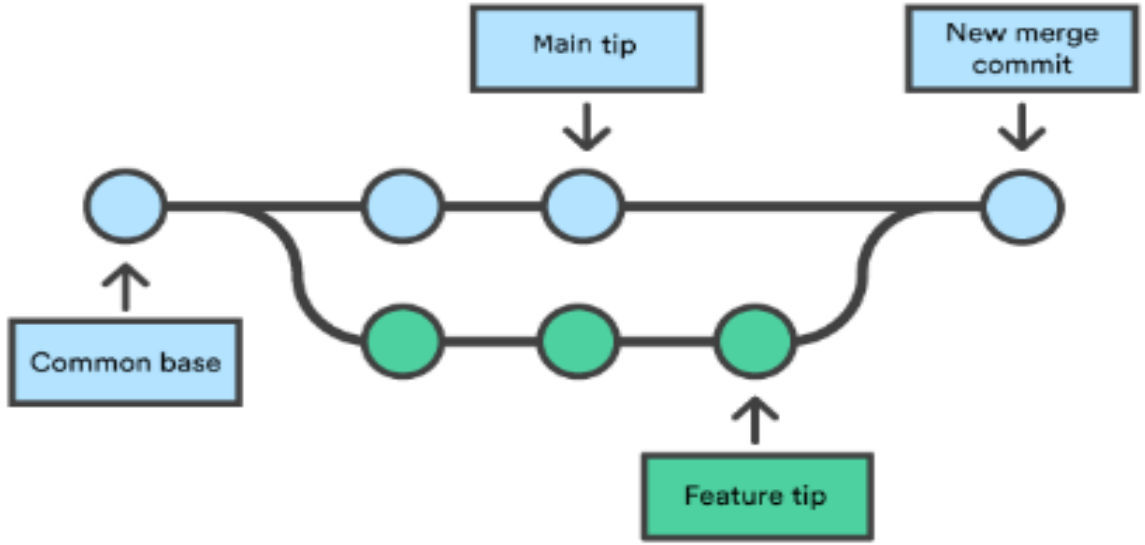
- La commande git merge est utilisée pour combiner deux branches.
- Autrement dit, pour fusionner deux branches, nous devons se placer dans la branche dans laquelle nous souhaitons fusionner puis effectuer un git merge.
- git merge prend deux pointeurs de commit, généralement les pointes de la branche, et recherchera un commit de base commune aux deux.
- Dès que Git trouve un commit de base commune, il créera un nouveau « commit de merge » qui combine les changements de chaque séquence de commit de merge dans la file d'attente.

Git Merge

Fonctionnement (2)

- git merge permet de merger la fonctionnalité de branche spécifiée dans la branche actuelle, telle que main ou master.
- Git déterminera automatiquement l'algorithme de merge.





- Les commits de merge sont uniques par rapport aux autres commits, car ils ont deux commits parent.
- En créant un commit de merge, Git essaiera de merger automatiquement les historiques distincts.

Git Merge

❖ Confirmer la branche cible

➔ Exécutez la commande **git status ou git branch** pour vous assurer que HEAD pointe vers la branche recevant le merge correspondant.
➔ Si nécessaire, exécutez **git checkout** pour passer à la branche cible.
➔ Dans notre cas, nous exécuterons **git checkout master**.

❖ Merge

➔ Après avoir exécuté les étapes de « préparation au merge » déjà interprétées, un merge peut être initié en exécutant **git merge** où désigne le nom de la branche qui sera mergée dans la branche cible.

Git Merge

Exemple de fusionnement

Pour fusionner des branches, on doit suivre les étapes suivantes :

1- Créer et basculer vers la branche feature et créez 3 commits

```
$ touch file.txt
$ git add file.txt
$ git commit -m « commit de base"
```

```
$ git checkout -b feature
$ echo 'résoudre le problème 1' >> featurefile.txt
$ git add featurefile.txt
$ git commit -m "correction 1 dans la branche"
$ echo 'résoudre le problème 2' >> featurefile.txt
$ git add featurefile.txt
$ git commit -m "correction 2 dans la branche"
$ echo 'résoudre le problème 3' >> featurefile.txt
$ git add featurefile.txt
$ git commit -m "correction 3 dans la branche"
```

Git Merge

Exemple de fusionnement

2- Passez à la branche principale (master) et créez 2 commits

```
$ git checkout master
$ echo ' traiter la solution 1' >> masterfile.txt
$ git add masterfile.txt
$ git commit -m " traitement 1 dans la branche"
$ echo ' traiter la solution 2' >> masterfile.txt
$ git add masterfile.txt
$ git commit -m " traitement 2 dans la branche"
```

3- Dans la même branche principale (master), fusionnez la modification des corrections dans le code source principal :

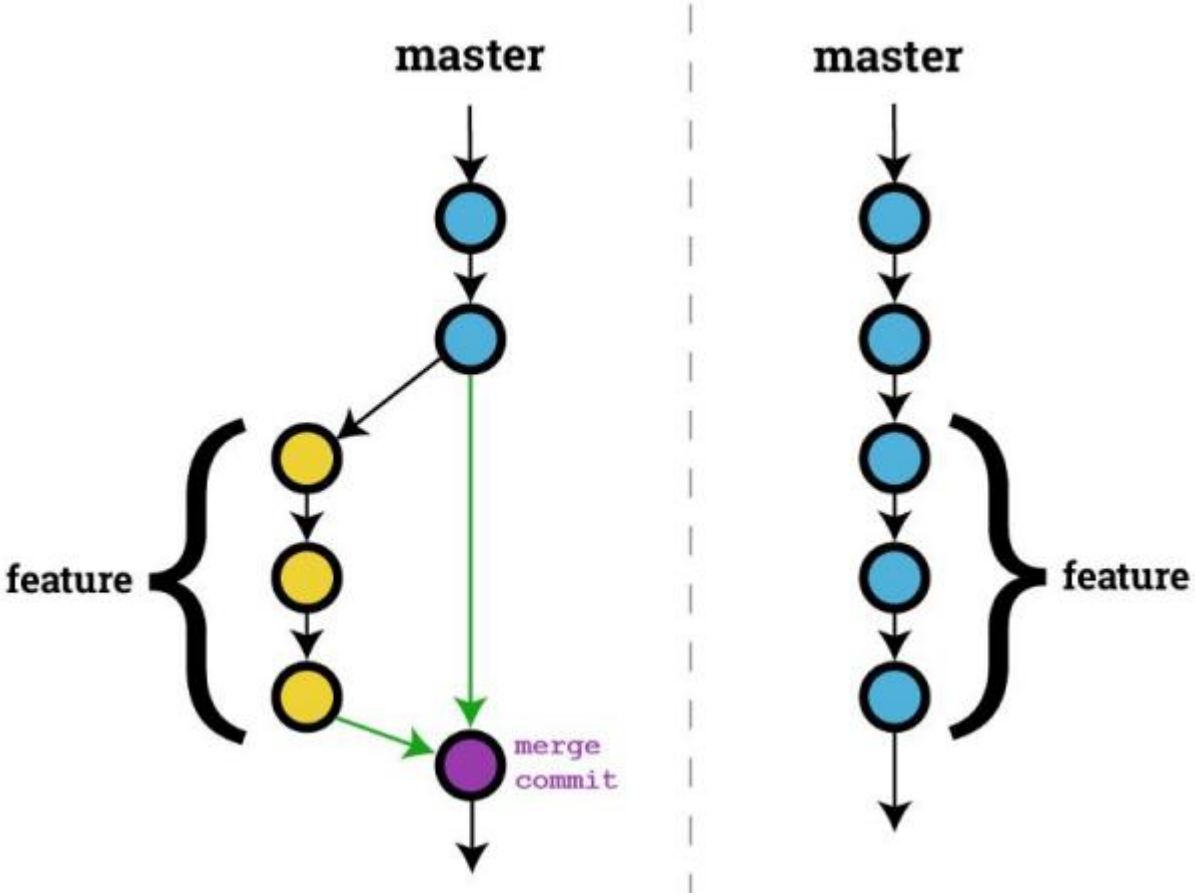
```
$ git merge feature // fusionner les commits de la
branche feature dans la branche master
```

Git Merge

➔ Fusionnement des branches et établissement d'un graphe de fusion des commits :

$ git log --oneline --graph --decorate

Exemple de fusionnement



Git Merge

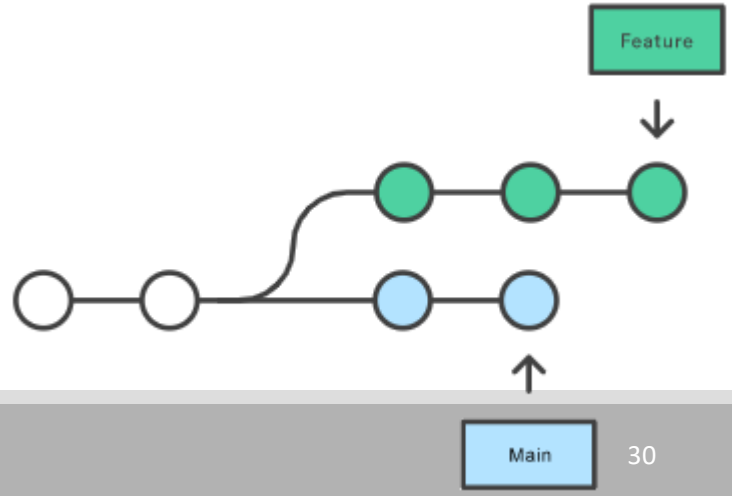
git rebase - git merge - git merge --squash

- La première chose à savoir sur la commande git rebase est qu'elle poursuit le même objectif que git merge.

- Ces deux commandes permettent de transférer des changements d'une branche à une autre. Seule la manière de procéder diffère.

- Imaginons que les nouveaux commits de la branche (main) soient pertinents pour la fonctionnalité sur laquelle nous travaillons.

- Deux méthodes seront interprétées pour ajouter les nouveaux commits dans la branche de fonctionnalité (feature) : le merge ou le rebase.



Git Merge

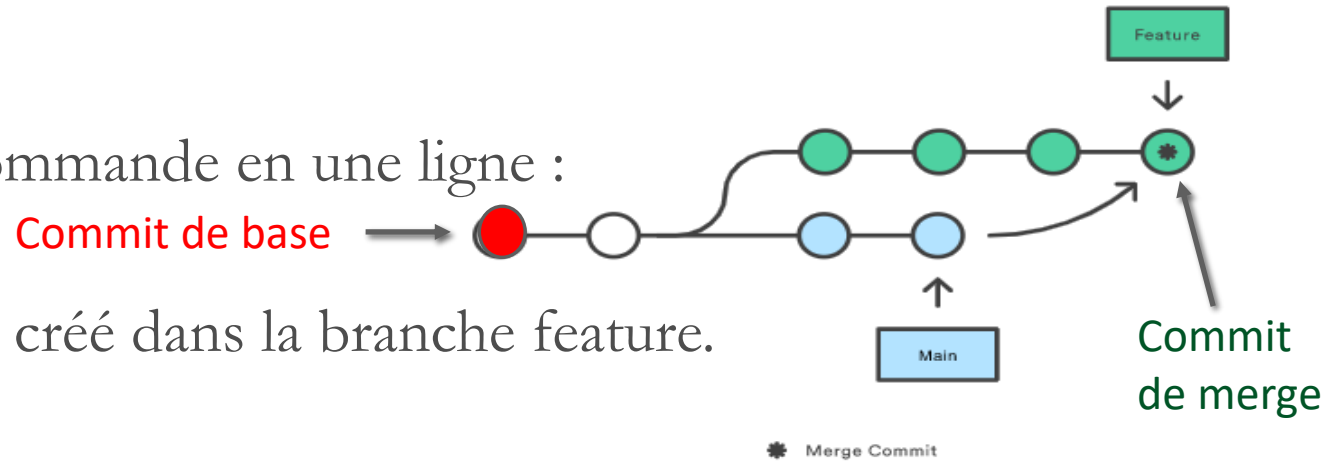
L'option Merge

- La solution la plus simple consiste à faire un merge de la branche principale (main ou master) dans la branche de fonctionnalité (feature) en procédant de la manière suivante :

```
$ git checkout feature
$ git merge master
```

- Nous pouvons également condenser cette commande en une ligne :

$ git merge feature master



- Un « commit de merge » (merge commit) est créé dans la branche feature.
- Il relie les historiques des deux branches.
- La structure de branche est alors redéfinie comme suit :

➔ Le merge est une opération **intéressante**, car elle est **non destructive**. Les branches existantes ne sont aucunement altérées.

➔ Cela permet d'éviter les pièges potentiels du rebase

Git Merge

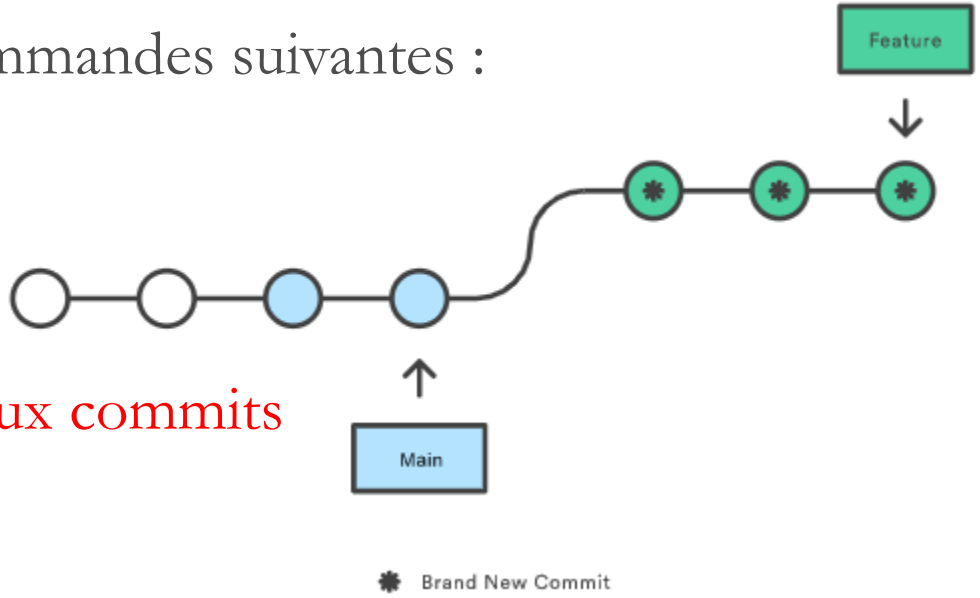
L'option Rebase

git rebase - git merge - git merge --squash

❖ Comme son nom l'indique, rebase est là pour changer la « base » d'une branche, c'est-à-dire son point de départ.

❖ Plutôt que de faire un merge, nous pouvons faire un rebase de la branche de fonctionnalité (feature) sur la branche (main ou master) en exécutant les commandes suivantes :

```
$ git checkout feature
$ git rebase master
```



❖ Toute la branche de fonctionnalité (feature) sera ainsi déplacée sur la pointe de la branche (main), et tous les nouveaux commits seront intégrés à main.

➔ Cependant, au lieu d'utiliser un commit de merge, le rebase consiste à réécrire l'historique du projet en créant de nouveaux commits pour chaque commit de la branche d'origine.

Git Merge

git rebase - git merge - git merge --squash

L'option Rebase

Le principal avantage du rebase est que l'historique de notre projet sera nettement plus propre.

➔ Premièrement, cette opération permet de supprimer les commits de merge superflus requis par la commande git merge.

➔ Deuxièmement, nous obtenons un historique de projet parfaitement linéaire, qui nous permettra de suivre la pointe de la branche de fonctionnalité à toutes les étapes du projet à partir de son commencement.

Git Merge

L'option git merge --squash

- La commande git merge --squash est une option de fusion qui nous permet de condenser l'historique Git des branches thématiques lorsque nous remplissons une demande d'extraction.
- Au lieu que chaque commit sur la branche thématique soit ajouté à l'historique de la branche par défaut, une fusion squash ajoute toutes les modifications du fichier à un seul nouveau commit sur la branche par défaut.
- Le commit de Merge Squash n'a pas de référence à la branche thématique, il produira un **nouveau commit** contenant toutes les modifications de la branche thématique.
- De plus, il est recommandé de supprimer la branche thématique pour éviter toute confusion.

Git Merge

L'option git merge --squash

- La fusion conserve l'historique des commits tel quel.
- Donc la branche dans laquelle nous avions exécuté la fusion sera mal organisé.

$ git merge –squash feature

$ git status

$ git commit –m “commit grouppant tous les commits de la branche feature”

Git Merge

git rebase - git merge - git merge --squash

On peut annuler la fusion (merge) des branches

$ git reset --hard HEAD~1 % pour revenir au commit avant le merge

$ git log ou git log --oneline

$ Git reflog